



CHAPTER 7

TYPES OF DATA STRUCTURES

Overview

- ▶ Types of Data Structure
 - ▶ Static Data Structure
 - ▶ Dynamic Data Structure
 - ▶ Linear Data Structure
 - ▶ Non-linear Data Structure

Static Data Structure

TYPES OF DATA
STRUCTURE

Data Structures

- ▶ Data structure is a way of storing and organizing data efficiently such that the required operations on them can be performed efficiently with respect to time as well as memory.
- ▶ Simply, Data Structure are used to reduce complexity (mostly the time complexity) of the code.
- ▶ Data structures can be of two types :
 - ▶ Static Data Structure
 - ▶ Dynamic Data Structure

Static Data Structure

- ▶ A static data structure is an organization or collection of data in memory that is fixed in size.
- ▶ Maximum size needs to be known in advance as memory cannot be reallocated at a later point.
- ▶ Example: Arrays.

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9
First Index = 0
Last Index = 8

Static Data Structure

- ▶ Are ideal for storing a fixed number of data items
- ▶ Lack the dynamic data structure's flexibility to use additional memory if needed or free up memory when possible for improved efficiency.
- ▶ Static data structures focus more on performance
- ▶ Dynamic data structures focus more on efficient memory usage

Advantage

- ▶ A key advantage of static data structures is that with memory allocation fixed, no control or oversight is needed to prevent potential overflow or underflow issues when adding new items or removing existing ones.
- ▶ This makes static data structures easier to program but at the expense of potential efficiency in terms of memory usage.

Dynamic Data Structure

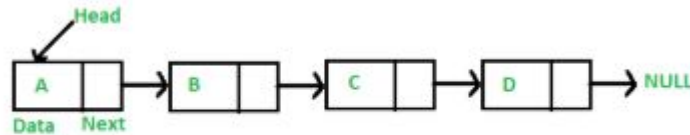
TYPES OF DATA
STRUCTURE

Dynamic Data Structure

- ▶ A dynamic data structure (DDS) refers to an organization or collection of data in memory that has the flexibility to grow or shrink in size, enabling a programmer to control exactly how much memory is utilized.
- ▶ Dynamic data structures change in size by having unused memory allocated or de-allocated from the heap as needed.

Dynamic Data Structure

- ▶ Dynamic data structures provide the programmer with the flexibility to adjust the memory consumption of software programs.
- ▶ Example: Linked List

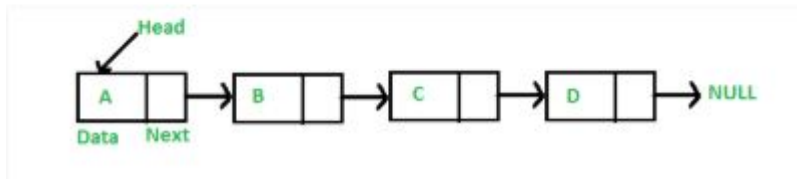
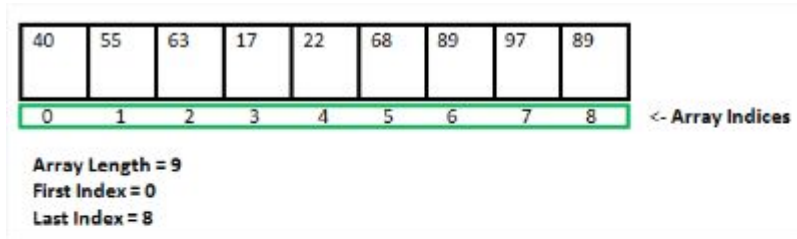


Dynamic Data Structure

- ▶ When the number of data items can't be predicted beforehand, a dynamic data structure should be used.
- ▶ A potential drawback to using dynamic data structures, though, is that because allocation of memory isn't fixed, there is the possibility for the structure to overflow if it exceeds the maximum allowed memory limit or underflow if the data structure becomes empty.
- ▶ To help prevent these issues from occurring, the programmer needs to add control for continually monitoring the size and location of data items in a dynamic data structure.

Linked List vs Array

- Both Arrays and Linked List can be used to store linear data of similar types, but they both have some advantages and disadvantages over each other.



Key Differences Between Array and Linked List

- ▶ An array is the data structure that contains a collection of similar type data elements whereas the Linked list is considered as non-primitive data structure contains a collection of unordered linked elements known as nodes.
- ▶ In the array the elements belong to indexes, i.e., if you want to get into the fourth element you have to write the variable name with its index or location within the square bracket.

Key Differences Between Array and Linked List

- ▶ In a linked list, you have to start from the head and work your way through until you get to the fourth element.
- ▶ Accessing an element in an array is fast, while Linked list takes linear time, so it is quite a bit slower.

Key Differences Between Array and Linked List

- ▶ Operations like insertion and deletion in arrays consume a lot of time. On the other hand, the performance of these operations in Linked lists is fast.
- ▶ Arrays are of fixed size. In contrast, Linked lists are dynamic and flexible and can expand and contract its size.

Key Differences Between Array and Linked List

- ▶ In an array, memory is assigned during compile time while in a Linked list it is allocated during execution or runtime.
- ▶ Elements are stored consecutively in arrays whereas it is stored randomly in Linked lists.

Key Differences Between Array and Linked List

- ▶ The requirement of memory is less due to actual data being stored within the index in the array. As against, there is a need for more memory in Linked Lists due to storage of additional next and previous referencing elements.
- ▶ In addition memory utilization is inefficient in the array. Conversely, memory utilization is efficient in the linked list.

Advantages of Linked Lists over Arrays

- ▶ The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage, and in practical uses, the upper limit is rarely reached.
- ▶ Inserting a new element in an array of elements is expensive because a room has to be created for the new elements and to create room existing elements have to be shifted.

Advantages of Linked Lists over Arrays

- ▶ For example, suppose we maintain a sorted list of IDs in an array `id[]`.

`id[] = [1000, 1010, 1050, 2000, 2040, .....]`

- ▶ And if we want to insert a new ID 1005, then to maintain the sorted order, we have to move all the elements after 1000 (excluding 1000).

Advantages of Linked Lists over Arrays

`id[] = [1000, 1010, 1050, 2000, 2040, .....]`

- ▶ Deletion is also expensive with arrays until unless some special techniques are used. For example, to delete 1010 in `id[]`, everything after 1010 has to be moved.

Advantages of Linked Lists over Arrays

- ▶ Linked list provides the following two advantages over arrays
 1. Dynamic size
 2. Ease of insertion/deletion

Disadvantages

- ▶ Linked lists have following drawbacks:
 1. Random access is not allowed. We have to access elements sequentially starting from the first node. So we cannot do a binary search with linked lists.
 2. Extra memory space for a pointer is required with each element of the list.
 3. Arrays have better cache locality that can make a pretty big difference in performance.

Cache Locality

- ▶ Arrays are contiguous memory blocks, so large chunks of them will be loaded into the cache upon first access.
- ▶ This makes it comparatively quick to access future elements of the array.
- ▶ Linked lists on the other hand aren't necessarily in contiguous blocks of memory, and could lead to more cache misses, which increases the time it takes to access them.

Cache Locality

- Consider the following possible memory layouts for an array data and linked list

l_data of large structs

1	Address	Contents	Address	Contents
2	aaaa 0000	data[0]	aaaa 1000	l_data
3				
4	aaaa 0040	data[1]	aaaa 3040	l_data->next
5				
6	aaaa 0080	data[2]	aaaa 4050	l_data->next->next

Cache Locality

- ▶ If we wanted to loop through this array, the first access to `aaaa_0000` would require us to go to memory to retrieve (a very slow operation in CPU cycles).
- ▶ However, after the first access the rest of the array would be in the cache, and subsequent accesses would be much quicker.

Cache Locality

- ▶ With the linked list, the first access to `aaaa 1000` would also require us to go to memory.
- ▶ Unfortunately, the processor will cache the memory directly surrounding this location, say all the way up to `aaaa 2000`.
- ▶ As you can see, this doesn't actually capture any of the other elements of the list, which means that when we go to access `l_data->next`, we will again have to go to memory.

Dynamic Data Structures	Static Data Structures
Memory is allocated to the data structure dynamically i.e. as the program executes.	Memory is allocated at compile time. Fixed size.
Because the memory allocation is dynamic, it is possible for the structure to 'overflow' should it exceed its allowed limit. It can also 'underflow' should it become empty.	The memory allocation is fixed and so there will be no problem with adding and removing data items
Makes the most efficient use of memory as the data structure only uses as much memory as it needs	Can be very inefficient as the memory for the data structure has been set aside regardless of whether it is needed or not whilst the program is executing.
Harder to program as the software needs to keep track of its size and data item locations at all times	Easier to program as there is no need to check on data structure size at any point.

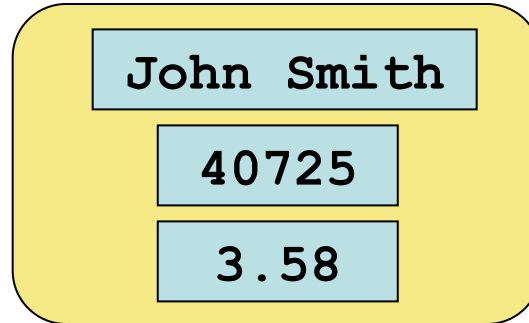
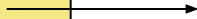
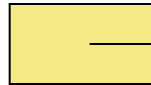
Linked List implementation

TYPES OF DATA
STRUCTURE

Object References

- ▶ Recall that an object reference is a variable that stores the address of an object
- ▶ A reference also can be called a pointer
- ▶ References often are depicted graphically:

student



References as Links

- ▶ Object references can be used to create links between objects
- ▶ Suppose a class contains a reference to another object of the same class:

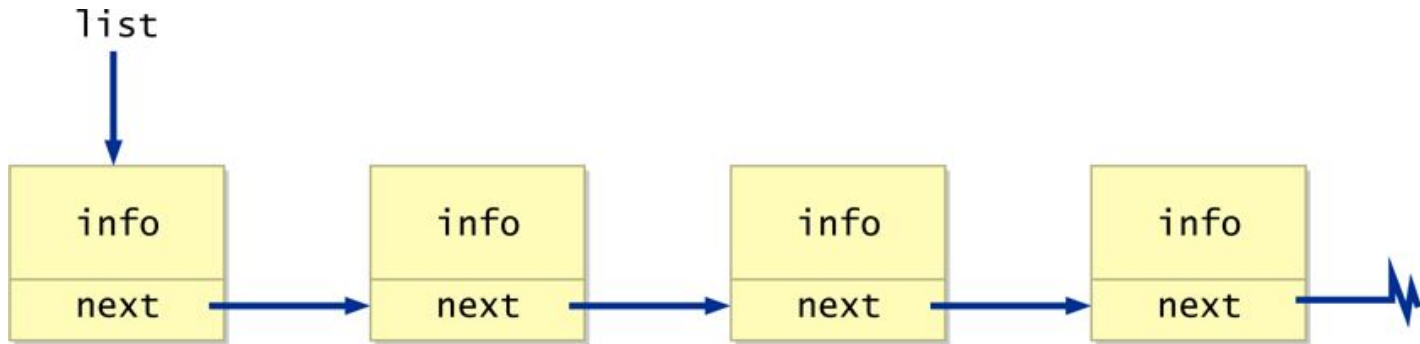
```
class Node  
{  
    int info;  
    Node next;  
}
```

Intermediate Nodes

- ▶ The objects being stored should not be concerned with the details of the data structure in which they may be stored
- ▶ For example, the Student class should not have to store a link to the next Student object in the list
- ▶ Instead, use a separate node class with two parts:
 - ▶ a reference to an independent object
 - ▶ a link to the next node in the list
- ▶ The internal representation becomes a linked list of nodes

References as Links

- References can be used to create a variety of linked structures, such as a linked list:



Magazine Implementation

- ▶ Let's explore an example of a collection of Magazine objects, managed by the MagazineList class, which has an private inner class called MagazineNode
- ▶ See MagazineRack.java
- ▶ See MagazineList.java
- ▶ See Magazine.java

```
//*****  
// MagazineRack.java    Author: Lewis/Loftus  
//  
// Driver to exercise the MagazineList collection.  
//*****
```

```
public class MagazineRack  
{  
    //-----  
    // Creates a MagazineList object, adds several magazines to the  
    // list, then prints it.  
    //-----  
    public static void main (String[] args)  
    {  
        MagazineList rack = new MagazineList();  
  
        rack.add (new Magazine("Time"));  
        rack.add (new Magazine("Woodworking Today"));  
        rack.add (new Magazine("Communications of the ACM"));  
        rack.add (new Magazine("House and Garden"));  
        rack.add (new Magazine("GQ"));  
  
        System.out.println (rack);  
    }  
}
```

```
//*****  
// MagazineRack.java      Author: Lewis/Loftus  
//  
// Driver to exercise the MagazineList collection.  
//*****
```

```
public class MagazineRack  
{  
    //-----  
    // Creates a MagazineList object, adds several magazines to the  
    // list, then prints it.  
    //-----  
    public static void main (String[] args)  
    {  
        MagazineList rack = new MagazineList();  
  
        rack.add (new Magazine("Time"));  
        rack.add (new Magazine("Woodworking Today"));  
        rack.add (new Magazine("Communications of the ACM"));  
        rack.add (new Magazine("House and Garden"));  
        rack.add (new Magazine("GQ"));  
  
        System.out.println (rack);  
    }  
}
```

Output

```
Time  
Woodworking Today  
Communications of the ACM  
House and Garden  
GQ
```

```
//*****  
// MagazineList.java    Author: Lewis/Loftus  
//  
// Represents a collection of magazines.  
//*****
```

```
public class MagazineList  
{  
    private MagazineNode list;
```

```
    //-----  
    // Sets up an initially empty list of magazines.  
    //-----
```

```
    public MagazineList()  
    {  
        list = null;  
    }
```

continue

continue

```
//-----  
// Creates a new MagazineNode object and adds it to the end of  
// the linked list.
```

```
//-----  
public void add (Magazine mag)  
{  
    MagazineNode node = new MagazineNode (mag);  
    MagazineNode current;  
  
    if (list == null)  
        list = node;  
    else  
    {  
        current = list;  
        while (current.next != null)  
            current = current.next;  
        current.next = node;  
    }  
}
```

continue

continue

```
//-----  
// Returns this list of magazines as a string.  
//-----  
public String toString ()  
{  
    String result = "";  
  
    MagazineNode current = list;  
  
    while (current != null)  
    {  
        result += current.magazine + "\n";  
        current = current.next;  
    }  
  
    return result;  
}
```

continue

continue

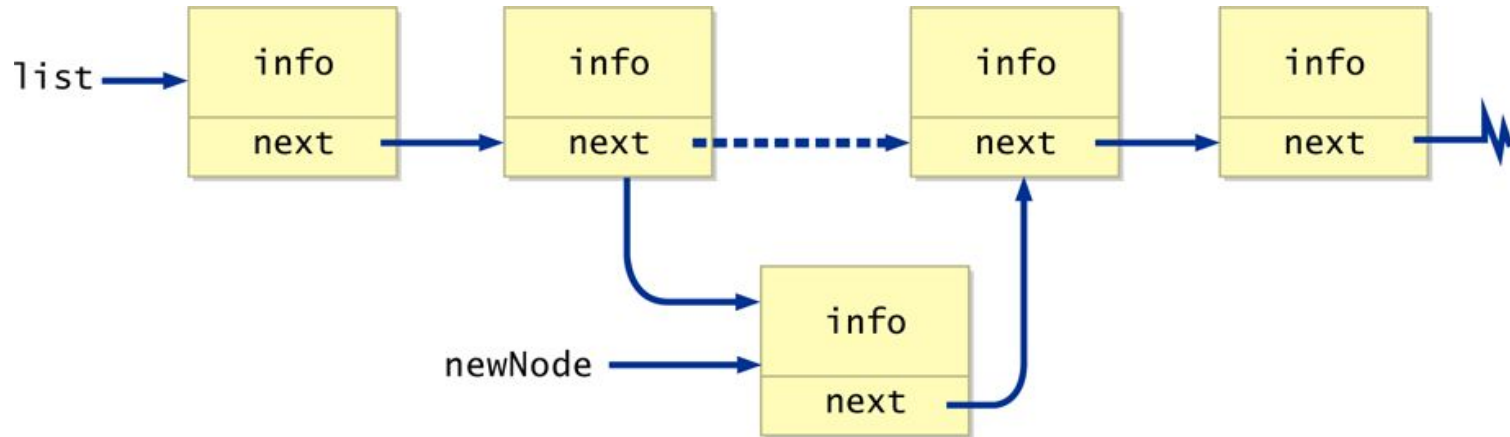
```
//*****  
// An inner class that represents a node in the magazine list.  
// The public variables are accessed by the MagazineList class.  
//*****  
private class MagazineNode  
{  
    public Magazine magazine;  
    public MagazineNode next;  
  
    //-----  
    // Sets up the node  
    //-----  
    public MagazineNode (Magazine mag)  
    {  
        magazine = mag;  
        next = null;  
    }  
}  
}
```

```
//*****  
// Magazine.java    Author: Lewis/Loftus  
//  
// Represents a single magazine.  
//*****
```

```
public class Magazine  
{  
    private String title;  
  
    //-----  
    // Sets up the new magazine with its title.  
    //-----  
    public Magazine (String newTitle)  
    {  
        title = newTitle;  
    }  
  
    //-----  
    // Returns this magazine as a string.  
    //-----  
    public String toString ()  
    {  
        return title;  
    }  
}
```

Inserting a Node

- ▶ A node can be inserted into a linked list with a few reference changes:



Quick Check

- ▶ Write code that inserts `newNode` after the node pointed to by `current`.

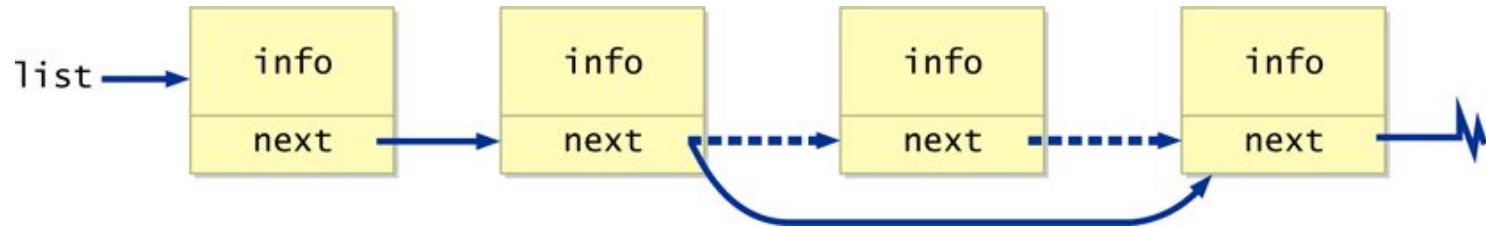
Quick Check

- ▶ Write code that inserts `newNode` after the node pointed to by `current`.

```
newNode.next = current.next;  
current.next = newNode;
```

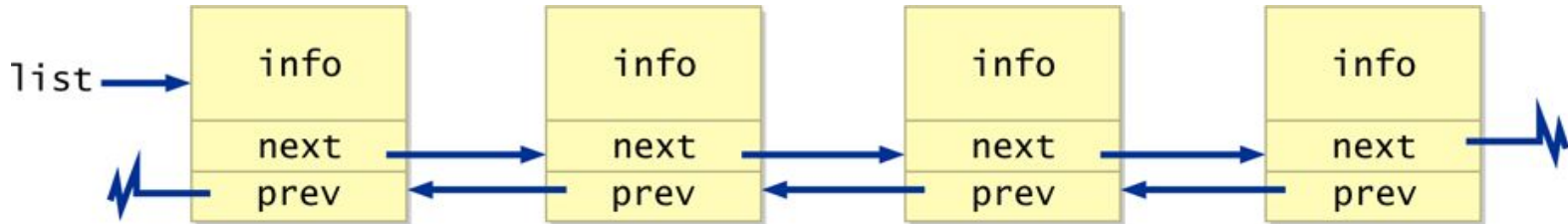
Deleting a Node

- ▶ Likewise, a node can be removed from a linked list by changing the next pointer of the preceding node:



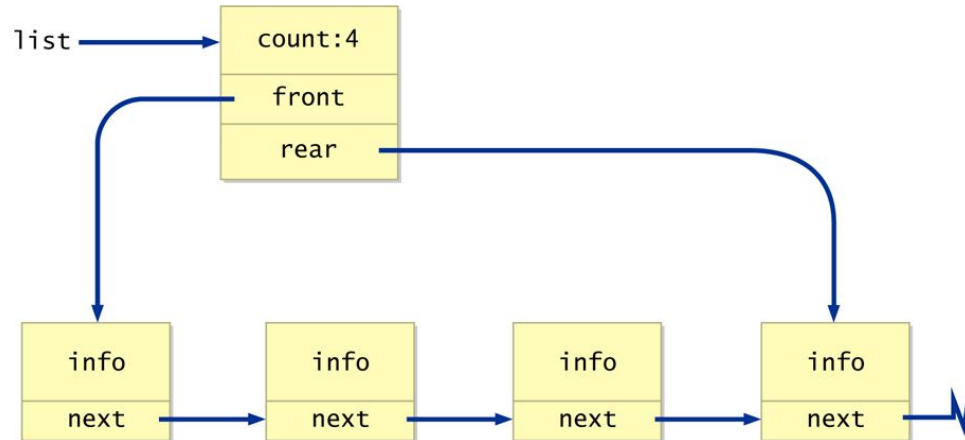
Other Dynamic Representations

- It may be convenient to implement a list as a doubly linked list, with next and previous references:



Other Dynamic Representation

- ▶ Another approach is to use a separate header node, with a count and references to both the front and rear of the list:



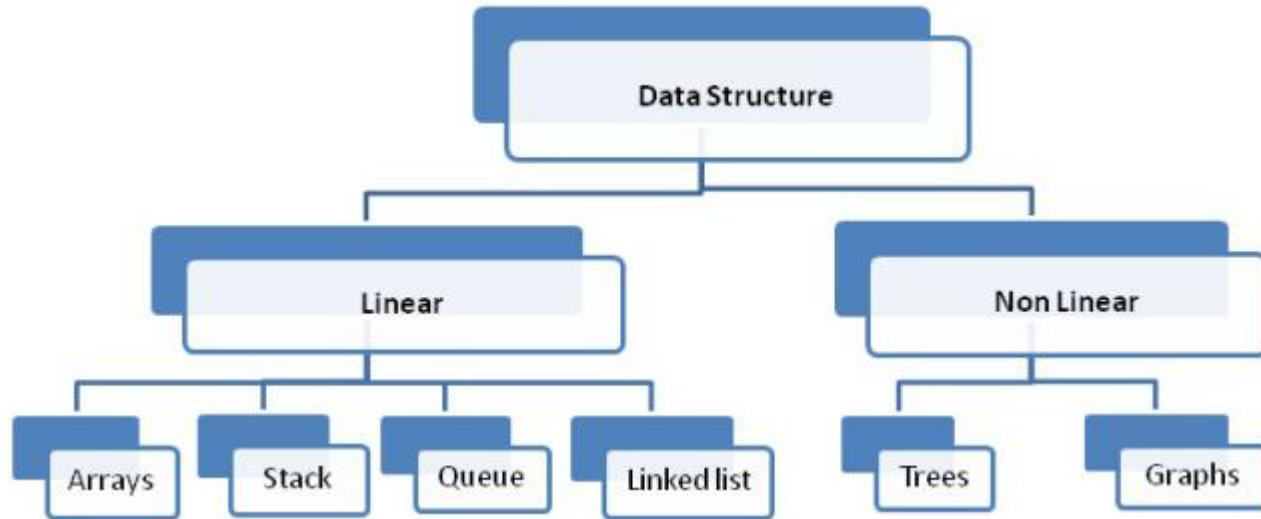
Linear Data Structure

TYPES OF DATA
STRUCTURE

Linear Data Structures

- ▶ Data elements are organized in a linear fashion, where each data element attaches one after the other.
- ▶ Data elements are traversed one after the other and only one element can be directly reached while traversing.
- ▶ Easy to implement, since the organization of the computer memory is also in a linear fashion.

Linear Data Structure



Linear Data Structures

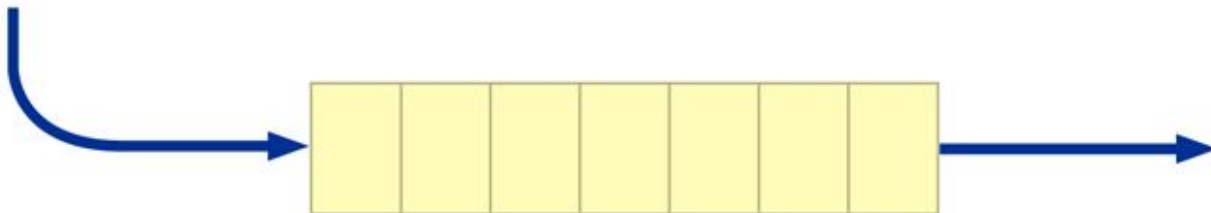
- ▶ Some commonly used linear data structures are arrays, linked lists, stacks, and queues.
- ▶ An array is a collection of data elements of the same type. The index helps to identify each element in the array.
- ▶ A linked list is a sequence of nodes, where each node is made up of a data element and a reference to the next node in the sequence.
- ▶ A stack is also a linear data structure. It is only possible to add or remove data elements from the top of the list.
- ▶ A queue is also a list. It allows addition of data elements from one end of the list and removal from the other end of the list.

Queues

- ▶ A queue is a list that adds items only to the rear of the list and removes them only from the front
- ▶ It is a FIFO data structure: First-In, First-Out
- ▶ Analogy: a line of people at a bank teller's window

Items go on the queue
at the rear (enqueue)

Items come off the queue
at the front (dequeue)



Queues

- ▶ Classic operations for a queue
 - ▶ enqueue - add an item to the rear of the queue
 - ▶ dequeue (or serve) - remove an item from the front of the queue
 - ▶ empty - returns true if the queue is empty
- ▶ Queues often are helpful in simulations or any situation in which items get “backed up” while awaiting processing

Queues

- ▶ A queue can be represented by a singly-linked list; it is most efficient if the references point from the front toward the rear of the queue
- ▶ A queue can be represented by an array, using the remainder operator (%) to “wrap around” when the end of the array is reached and space is available at the front of the array

Queues – Direct Applications

- ▶ Operating systems schedule jobs (with equal priority) in the order of arrival (e.g., a print queue).
- ▶ Simulation of real-world queues such as lines at a ticket counter or any other first-come first-served scenario requires a queue.
- ▶ Multiprogramming.
- ▶ Asynchronous data transfer (file IO, pipes, sockets).
- ▶ Waiting times of customers at call center.
- ▶ Determining number of cashiers to have at a supermarket.

Queues – Indirect Applications

- ▶ Auxiliary data structure for algorithms
- ▶ Component of other data structures

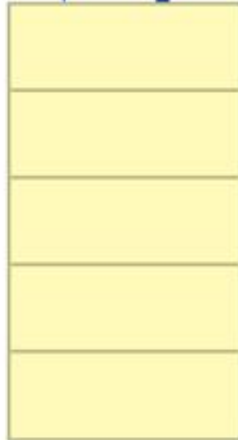
Stacks

- ▶ A stack ADT is also linear, like a list or a queue
- ▶ Items are added and removed from only one end of a stack
- ▶ It is therefore LIFO: Last-In, First-Out
- ▶ Analogies: a stack of plates or a stack of books

Stacks

▶ The last item to go
on the stack (push)

must be the first item
to come off (pop)



Stacks

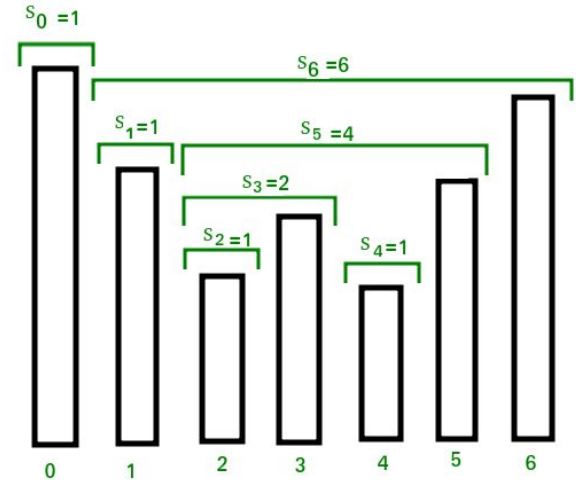
- ▶ Classic stack operations:
 - ▶ push - add an item to the top of the stack
 - ▶ pop - remove an item from the top of the stack
 - ▶ peek (or top) - retrieves the top item without removing it
 - ▶ empty - returns true if the stack is empty
- ▶ A stack can be represented by a singly-linked list, with the first node in the list being the top element on the stack
- ▶ A stack can also be represented by an array, with the bottom of the stack at index 0

Stacks – Direct Applications

- ▶ Balancing of symbols
- ▶ Infix-to-postfix conversion
- ▶ Evaluation of postfix expression
- ▶ Implementing function calls (including recursion)
- ▶ Finding of spans (finding spans in stock markets, refer to Problems section)
- ▶ Page-visited history in a Web browser [Back Buttons]
- ▶ Undo sequence in a text editor
- ▶ Matching Tags in HTML and XML

The Stock Span Problem

- ▶ The stock span problem is a financial problem where we have a series of n daily price quotes for a stock and we need to calculate span of stock's price for all n days.
- ▶ The span S_i of the stock's price on a given day i is defined as the maximum number of consecutive days just before the given day, for which the price of the stock on the current day is less than or equal to its price on the given day.
- ▶ For example, if an array of 7 days prices is given as $\{100, 80, 60, 70, 60, 75, 85\}$, then the span values for corresponding 7 days are $\{1, 1, 1, 2, 1, 4, 6\}$



Stacks – Indirect Applications

- ▶ Auxiliary data structure for other algorithms (Example: Tree traversal algorithms)
- ▶ Component of other data structures

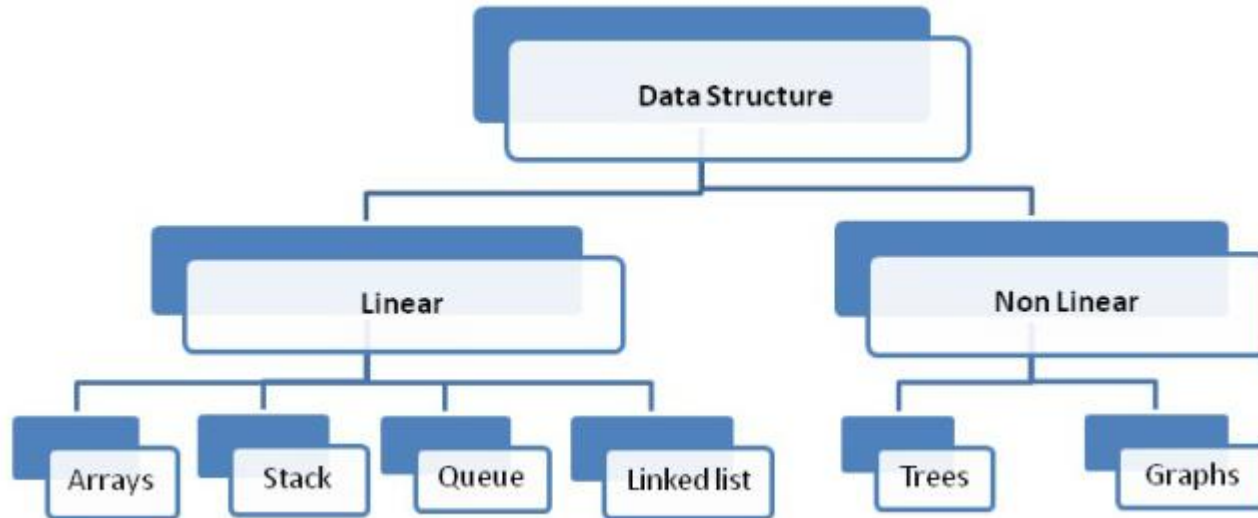
Non-linear Data Structure

TYPES OF DATA
STRUCTURE

Nonlinear Data Structures

- ▶ In nonlinear data structures, the organization of the data elements is not in a sequential fashion.
- ▶ It is possible to attach a data item in a nonlinear data structure to several other data elements to reflect a special relationship among them.
- ▶ It is not possible to traverse through the items in a single run.

Non Linear Data Structure



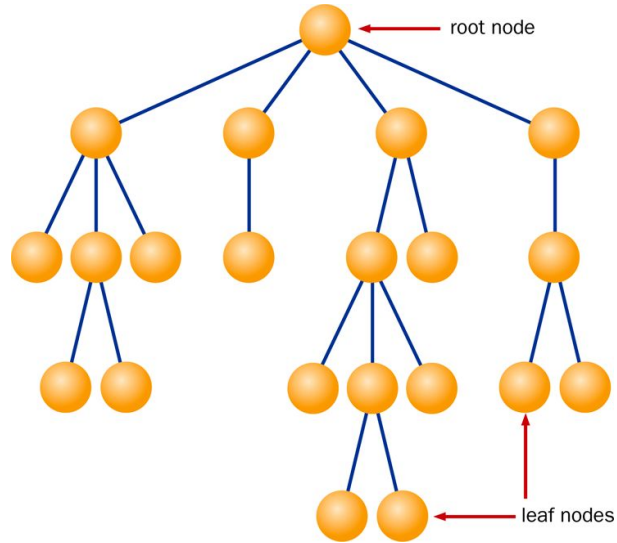
Nonlinear Data Structures

- ▶ Data structures like trees and graphs are two examples of nonlinear data structures.
- ▶ A tree is a data structure that is made up of a set of linked nodes. It allows representing a hierarchical relationship among data elements.
- ▶ A graph is a data structure that is made up of a finite set of edges and vertices. The vertices stored data elements and edges represent the connections or relationships among the vertices.

Trees

- ▶ A tree is a non-linear data structure that consists of a root node and potentially many levels of additional nodes that form a hierarchy
- ▶ Nodes that have no children are called leaf nodes
- ▶ Nodes except for the root and leaf nodes are called internal nodes
- ▶ In a general tree, each node can have many child nodes

A General Tree



Binary Trees

- ▶ In a binary tree, each node can have no more than two child nodes
- ▶ Trees are typically represented using references as dynamic links
- ▶ For binary trees, this requires storing only two links per node to the left and right child

Applications of Binary Trees

- ▶ Following are the some of the applications where binary trees play an important role:
 - ▶ Expression trees are used in compilers.
 - ▶ Huffman coding trees that are used in data compression algorithms.
 - ▶ Binary Search Tree (BST), which supports search, insertion and deletion on a collection of items in $O(\log n)$ (average).
 - ▶ Priority Queue (PQ), which supports search and deletion of minimum (or maximum) on a collection of items in logarithmic time (in worst case).

Binary Tree Traversals

- ▶ In order to process trees, we need a mechanism for traversing them
- ▶ The process of visiting all nodes of a tree is called tree traversal.
- ▶ Each node is processed only once but it may be visited more than once.
- ▶ As we have already seen in linear data structures (like linked lists, stacks, queues, etc.), the elements are visited in sequential order.

Binary Tree Traversals

- ▶ There are many different ways to traverse a tree structure.
- ▶ Tree traversal is like searching the tree, except that in traversal the goal is to move through the tree in a particular order.
- ▶ In addition, all nodes are processed in the traversal but searching stops when the required node is found.

Binary Tree Traversals

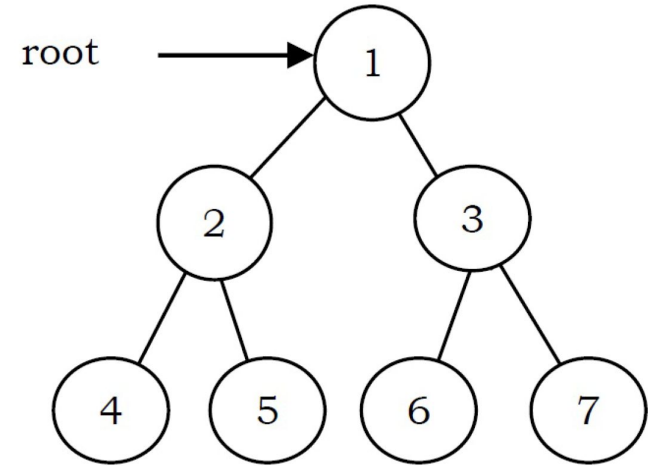
- ▶ Starting at the root of a binary tree, there are three main steps that can be performed and the order in which they are performed defines the traversal type.
- ▶ These steps are:
 - ▶ Performing an action on the current node (referred to as “visiting” the node and denoted with “D”),
 - ▶ Traversing to the left child node (denoted with “L”), and
 - ▶ Traversing to the right child node (denoted with “R”).
- ▶ This process can be easily described through recursion.

Binary Tree Traversals

- ▶ Three tree traversals methods:
 - ▶ Preorder (DLR) Traversal
 - ▶ Inorder (LDR) Traversal
 - ▶ Postorder (LRD) Traversal

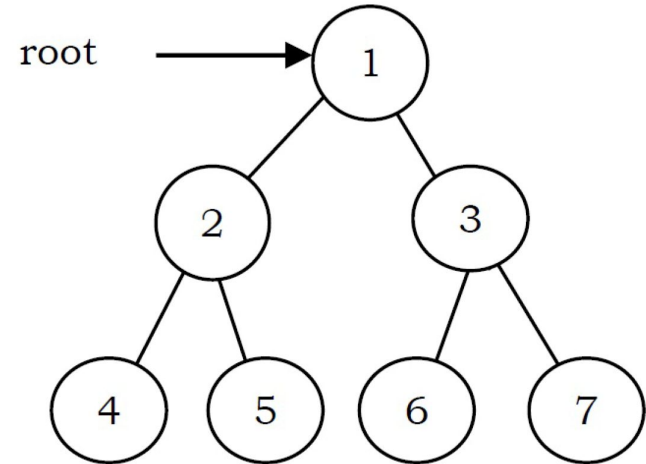
PreOrder Traversal

- ▶ Preorder traversal is defined as follows:
 - ▶ Visit the root.
 - ▶ Traverse the left subtree in Preorder.
 - ▶ Traverse the right subtree in Preorder.
- ▶ The nodes of tree would be visited in the order: 1 2 4 5 3 6 7



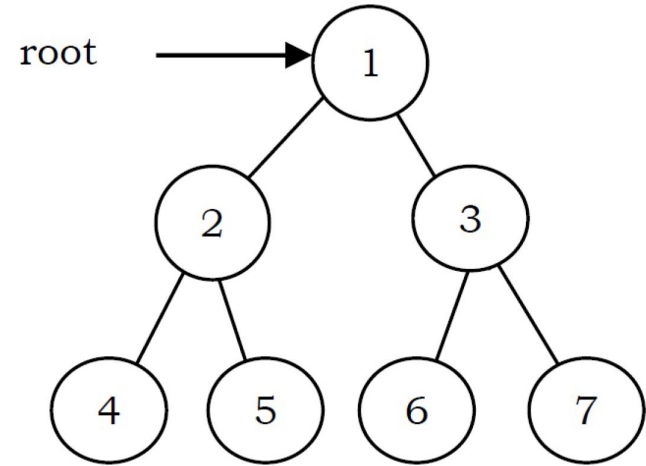
InOrder Traversal

- ▶ In Inorder Traversal the root is visited between the subtrees.
Inorder traversal is defined as follows:
 - ▶ Traverse the left subtree in Inorder.
 - ▶ Visit the root.
 - ▶ Traverse the right subtree in Inorder.
- ▶ The nodes of tree would be visited in the order: 4 2 5 1 6 3 7



PostOrder Traversal

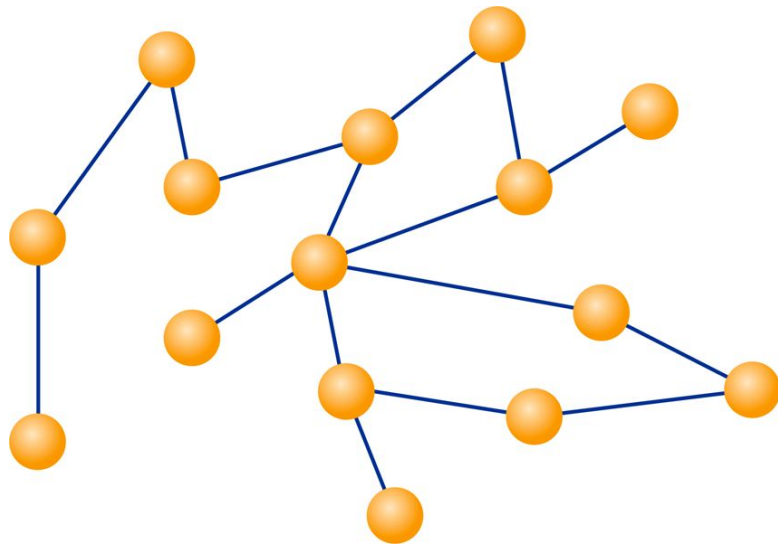
- ▶ In postorder traversal, the root is visited after both subtrees. Postorder traversal is defined as follows:
 - ▶ Traverse the left subtree in Postorder.
 - ▶ Traverse the right subtree in Postorder.
 - ▶ Visit the root.
- ▶ The nodes of the tree would be visited in the order: 4 5 2 6 7 3 1



Graphs

- ▶ A graph is another non-linear structure
- ▶ Unlike a tree, a graph does not have a root
- ▶ Any node in a graph can be connected to any other node by an edge
- ▶ Analogy: the highway system connecting cities on a map

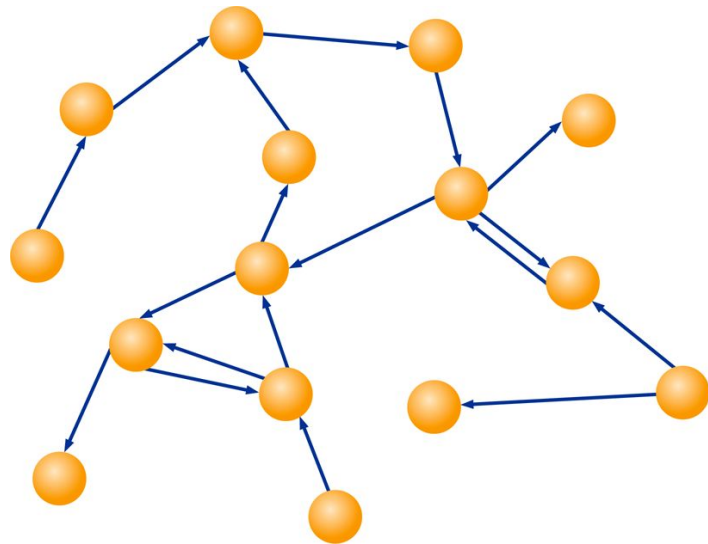
Graphs



Graphs

- ▶ In a directed graph or digraph, each edge has a specific direction.
- ▶ Edges with direction sometimes are called arcs
- ▶ Analogy: airline flights between airports

Directed Graphs



Representing Graphs

- ▶ Both graphs and digraphs can be represented using dynamic links or using arrays.
- ▶ As always, the representation should facilitate the intended operations and make them convenient to implement

Differences between Linear and nonlinear Data Structures

Linear Data Structures	Nonlinear Data Structures
Data elements are organized in a sequential manner	Data elements are not organized in a sequential manner
It is possible to traverse the data elements in a single run	It not possible to traverse the data elements in a single run
Easier to implement	Difficult to implement
Examples: Array, Stack, Queue, Linked List	Examples: Tree and Graph

Summary

- ▶ Types of Data Structure
 - ▶ Static Data Structure (Arrays)
 - ▶ Dynamic Data Structure (Linked Lists)
 - ▶ Linear Data Structure (Queues, Stacks)
 - ▶ Non-linear Data Structure (Trees, Graphs)

END OF CHAPTER 7

TYPES OF DATA STRUCTURE